

Sliding Window Approach

Yes — **your sliding-window idea is exactly the right approach**. The important part is that you should **sort the array first**, and then expand the window from left to right while treating the **rightmost/largest element as the target**.

Core intuition

Suppose:

```
nums = [1, 2, 4]
k = 5
```

After sorting:

```
[1, 2, 4]
```

Take 4 as the target:

```
1 2 4
↑ ↑ ↑
```

To make all elements in the window equal to 4:

```
4 - 1 = 3
4 - 2 = 2
```

Total operations:

```
3 + 2 = 5
```

So frequency = 3.

Sliding window idea

For every right endpoint r :

```
[l ..... r]
```

We want to make **every element from l to r equal to $nums[r]$** .

The required operations are:

```
(nums[r] - nums[l])
+ (nums[r] - nums[l+1])
+ ...
+ (nums[r] - nums[r-1])
```

If this cost is:

```
<= k
```

then the whole window is possible, so:

```
C++
frequency = r - l + 1
```

If cost becomes:

```
> k
```

we cannot afford this window, so we move l forward.

One important correction to your idea

You said:

"check the difference and remove that difference from k "

That works conceptually, **but be careful**: when the window has multiple elements, you can't simply subtract one difference from k . The cost of the whole window matters.

For example:

```
[1, 2, 4]
```

For target 4:

```
4-1 + 4-2 = 3 + 2 = 5
```

So we calculate the **total cost of the window**.

A very clean way is to maintain the window's sum.

For window $[l \dots r]$:

```
cost = nums[r] * windowSize - windowSum
```

Because:

```
nums[r] * windowSize
```

is what the sum would be if everything became `nums[r]`.

Code

```
C++
class Solution {
public:
    int maxFrequency(vector<int>& nums, int k) {
        sort(nums.begin(), nums.end());

        long long windowSum = 0;
        int l = 0;
        int ans = 1;

        for (int r = 0; r < nums.size(); r++) {
            // Add current element to the window
            windowSum += nums[r];

            // Cost to make every element in [l...r]
            // equal to nums[r]
            long long cost =
                1LL * nums[r] * (r - l + 1) - windowSum;

            // If we don't have enough operations,
            // shrink the window from the left
            while (cost > k) {
                windowSum -= nums[l];
                l++;

                cost =
                    1LL * nums[r] * (r - l + 1) - windowSum;
            }

            // Current window is achievable
            ans = max(ans, r - l + 1);
        }

        return ans;
    }
};
```

Think of it like this

```
Sort
↓
Expand right
↓
Try making entire window = nums[right]
↓
Calculate required operations
↓
cost <= k ?
↓
YES → increase frequency
↓
NO → move left until affordable
```

The **key observation** is:

Because we can only increment elements, after sorting, if we want a group of elements to become equal, the best target is the **largest element in that group** (`nums[r]`).

And yes, this is a **sliding window + sorting + prefix/window sum** problem.

😂 Yeah, that formula looks confusing at first. Let's forget the formula for a second and understand **what we are actually calculating**.

Suppose:

```
nums = [1, 2, 4]
k = 5
```

We are currently considering the window:

```
[1, 2, 4]
```

We want to make **all of them equal to 4**.

So directly calculate the operations:

```
1 → 4 needs 3 operations
2 → 4 needs 2 operations
4 → 4 needs 0 operations
Total = 3 + 2 + 0 = 5
```

So:

```
cost = 5
```

Now where does this come from?

Instead of doing:

```
(4 - 1) + (4 - 2) + (4 - 4)
```

we can rearrange it:

```
= 4 + 4 + 4 - (1 + 2 + 4)
```

Look at this:

```
4 + 4 + 4
```

There are **3 elements**, and we want each one to become 4.

So:

```
4 * 3 = 12
```

That's where:

```
C++
nums[r] * windowSize
```

comes from.

And:

```
1 + 2 + 4 = 7
```

is our `windowSum`.

Therefore:

```
cost = 12 - 7
      = 5
```

Which is exactly:

```
(4-1) + (4-2) + (4-4)
= 3 + 2 + 0
= 5
```

So remember it simply

```
C++
cost = target * number_of_elements - sum_of_elements;
```

For:

```
[1, 2, 4]
target = 4
number of elements = 3
sum = 7
```

Therefore:

```
cost = 4 * 3 - 7
      = 5
```

That's **all the formula is doing**.

You don't need to memorize it. Just remember:

"What should the total sum be if everyone becomes the target?" – "What is the current total sum?" = operations needed.

Yes. The `long long` is needed **not because** `nums[i]` **itself is huge**, but because the **calculation can become larger than** `int`.

Why `long long`?

Constraints:

```
nums[i] <= 10^5
n <= 10^5
k <= 10^5
```

Individually, `nums[i]` is only **100,000**, so `int` is enough for each element.

But look at this calculation:

```
C++
nums[r] * (r - l + 1)
```

The maximum can be roughly:

```
100,000 × 100,000
= 10,000,000,000
= 10 billion
```

But a normal C++ `int` can hold only approximately:

```
-2.1 billion to +2.1 billion
```

So **10 billion does not fit inside** `int`.

That's why we use:

```
C++
long long
```

which can hold values up to around:

```
9 × 10^18
```

This part is especially important

If you write:

```
C++
long long cost = nums[r] * (r - l + 1) - windowSum;
```

it may already be too late.

Because:

```
C++
nums[r]
```

is an `int`, and

```
C++
(r - l + 1)
```

is an `int`.

So C++ can perform the multiplication as **int multiplication first**.

That's why we write:

```
C++
long long cost =
    1LL * nums[r] * (r - l + 1) - windowSum;
```

The `1LL` forces the calculation to happen using `long long`.

Simple rule to remember

Whenever you see something like:

```
C++
int x int
```

and the result can become very large:

```
C++
1LL * a * b
```

is a safe habit.

And yes, **the constraints are big enough to cause this:**

```
n      = 100,000
nums[i] = 100,000

100,000 x 100,000 = 10,000,000,000
```

So `long long` is necessary for the **window sum/cost calculation**, even though the individual values are small.

ChatGPT can make mistakes. Check important info.